

03-diagrams

03 — Diagrams extraction

Exhaustive extraction of every architecture diagram for the **BobaLink** browser-extension implementation. Source: /Volumes/T7/Projects/Boba/docs/research/browser_extension/Browser Torrent Extension Guide/diagrams/. Project name in diagrams = **BobaLink**.

Files read

#	Diagram	Source formats read	Notes
01	System architecture	.mmd + .drawio.xml	drawio adds Chrome APIs panel, Docker networks, legend flows
02	Data flow — torrent detection → download	.mmd	(no drawio/puml)
03	Data flow — tab-groups batch	.mmd + .drawio.xml	drawio is a stub (swimlanes only)
04	Sequence — single torrent send	.mmd + .puml	mmd has par, puml has alt (same two paths)
05	Sequence — tab-group batch send	.mmd + .puml	
06	Component diagram	.mmd + .puml	mmd lists per-file functions; puml is

#	Diagram	Source formats read	Notes
07	ER diagram (DATA MODEL)	.mmd + .puml + .drawio.xml	package-only mmd is authoritative (most fields); drawio drops a few fields
08	Extension lifecycle	.mmd	
09	Offline-queue state machine	.mmd	
10	Security architecture	.mmd + .drawio.xml	
11	Build pipeline	.mmd	
12	Compatibility matrix	.html (read in full)	no .mmd; only .html/.svg exist

.svg / rendered .html exports of 01-11 deliberately NOT read (redundant renders). 12-compatibility-matrix.html read because no text-source .mmd exists for it.

1) Manifest

See “Files read” table above.

2) COMPLETE DATA MODEL (ER diagram #07)

6 entities. The .mmd is authoritative (carries the most fields + key/comment annotations). PlantUML and drawio render fewer fields (gaps noted per-entity). Types below: .mmd uses generic (string/number/boolean/datetime/json); .puml/.drawio give SQL types (VARCHAR/TEXT/INTEGER/BIGINT/BOOLEAN/DATETIME/JSON).

Entity: extension_config (#07 — single per-install extension settings)

Field	Type (mmd / sql)	Key	Comment
config_id	string / VARCHAR	PK	PRIMARY KEY
server_url	string / VARCHAR		Boba server URL
api_key	string / VARCHAR		API authentication key
qbittorrent_url	string / VARCHAR		qBitTorrent WebUI URL
username	string / VARCHAR		qBitTorrent username
password_encrypted	string / VARCHAR		Encrypted password
auto_detect	boolean / BOOLEAN		Auto-detect torrents
default_category	string / VARCHAR		Default qBit category
scan_interval_ms	number / INTEGER		DOM scan interval
max_retries	number / INTEGER		Max retry attempts
retry_delay_ms	number / INTEGER		Base retry delay
created_at	datetime / DATETIME		Record creation time
updated_at	datetime / DATETIME		Last update time

Entity: discovered_torrents (#07 — torrents found on pages)

Field	Type	Key	Comment
torrent_id	string / VARCHAR	PK	UUID
infohash	string / VARCHAR	UK (unique)	Unique infohash
magnet_uri	string / TEXT		Full magnet URI
name			

Field	Type	Key	Comment
	string / VARCHAR		Torrent display name
size_bytes	number / BIGINT		Torrent size
source_url	string / VARCHAR		Page where found
source_tab_id	string / VARCHAR		Browser tab ID
status	string / VARCHAR		pending sent failed
discovered_at	datetime / DATETIME		Discovery timestamp
sent_at	datetime / DATETIME		Send timestamp
error_message	string / TEXT		Error if failed

| drawio omits source_tab_id. mmd is the superset.

Entity: download_queue (#07 — offline/retry queue)

Field	Type	Key	Comment
queue_id	string / VARCHAR	PK	UUID
torrent_id	string / VARCHAR	FK → discovered_torrents	References discovered
magnet_uri	string / TEXT		Magnet link
priority	number / INTEGER		Queue priority 0- 10
retry_count	number / INTEGER		Current retry count
status	string / VARCHAR		queued retrying failed completed
queued_at	datetime / DATETIME		Queue entry time
last_retry_at	datetime / DATETIME		Last retry timestamp
error_log	string / TEXT		JSON error history

| drawio omits last_retry_at. mmd is the superset.

Entity: server_config (#07 — one row per Boba server, multi-server support)

Field	Type	Key	Comment
server_id	string / VARCHAR	PK	UUID
name	string / VARCHAR		Server display name
fastapi_url	string / VARCHAR		FastAPI endpoint URL
go_service_url	string / VARCHAR		Go service endpoint URL
qbittorrent_url	string / VARCHAR		qBitTorrent WebUI URL
api_key	string / VARCHAR		Authentication key
is_active	boolean / BOOLEAN		Is this server active
health_check_interval	number / INTEGER		Health check ms
last_health_check	datetime / DATETIME		Last check timestamp
last_check_result	boolean / BOOLEAN		Last check passed
created_at	datetime / DATETIME		Creation time

| drawio omits last_health_check. mmd is the superset.

Entity: send_history (#07 — audit log of every send attempt)

Field	Type	Key	Comment
history_id	string / VARCHAR	PK	UUID
infohash	string / VARCHAR		Torrent infohash
magnet_uri	string / TEXT		Magnet link
name	string / VARCHAR		Torrent name
server_id			

Field	Type	Key	Comment
	string / VARCHAR	FK → server_config	References server_config
success	boolean / BOOLEAN		Send succeeded
qbittorrent_hash	string / VARCHAR		Hash from qBit
category	string / VARCHAR		qBit category used
tags	string / VARCHAR		qBit tags
sent_at	datetime / DATETIME		Send timestamp
response_time_ms	number / INTEGER		API response time
error_message	string / TEXT		Error if failed

drawio omits magnet_uri, qbittorrent_hash, tags. mmd is the superset.

Entity: search_cache (#07 — cached search results, TTL'd)

Field	Type	Key	Comment
cache_id	string / VARCHAR	PK	UUID
query_hash	string / VARCHAR		Hash of search query
query_text	string / VARCHAR		Original search text
results	json / JSON		Cached results JSON
result_count	number / INTEGER		Number of results
cached_at	datetime / DATETIME		Cache timestamp
ttl_seconds	number / INTEGER		Cache TTL
is_valid	boolean / BOOLEAN		Cache still valid

search_cache has NO declared relationship to any other entity (standalone).

Relationships (cardinality) (#07)

- extension_config ||--o{ discovered_torrents — one-to-many — label “references config”

- discovered_torrents ||--o{ download_queue — one-to-many — label “queued for retry” (FK download_queue.torrent_id)
- server_config ||--o{ send_history — one-to-many — label “sends tracked” (FK send_history.server_id)

(||--o{ = one mandatory ↔ zero-or-many.)

3) COMPONENT INVENTORY + CONNECTIONS

3a) System architecture components (#01)

Browser Environment: - *Extension UI Layer:* Popup UI (popup.html/popup.js — torrent list & controls), Options Page (options.html/options.js — server configuration), Context Menu (background.js — right-click actions) - *Extension Core Layer:* Content Script (content.js — DOM scanner & injector; drawio adds “Torrent detection UI”), Background Service Worker (background.js — event handling & API proxy; drawio adds “API proxy & queue”) - *Extension Library Layer:* parser/ (magnet:// & .torrent parser), scanner/ (DOM torrent detector), api/BobaAPIClient (HTTP client for services), shared/ (constants & utilities) - *Extension Storage:* Local Storage (extension_config), Session Storage (discovered_torrents) - *Chrome APIs Used* (drawio-only panel): chrome.runtime (messaging), chrome.tabs (tab management), chrome.tabGroups (group queries), chrome.contextMenus (right-click menu), chrome.storage (local & session), chrome.notifications (desktop alerts), fetch() API (HTTP requests), chrome.downloads (file downloads)

Docker Host: - *Boba Services Network:* Boba FastAPI (Port **7187** — metadata & search API; torrent info, search, health checks), Boba Go Service (Port **7189** — core torrent operations; add, remove, control) - *Torrent Network:* qBitTorrent WebUI (Port **8080** — torrent client API), qBitTorrent Database (resume data, settings, logs) - *Docker Networks* (drawio-only): boba-network (internal mesh: FastAPI↔Go↔qBit), bridge/host (external port mapping 7187, 7189, 8080), Volume Mounts (/downloads, /config)

External Services: BitTorrent Trackers (DHT, PEX, UDP announce; peer discovery & metadata; magnet resolution), Web Pages / Indexers (torrent sites & search engines; magnet hosting; .torrent download pages)

Connections (#01): - Popup ↔ BackgroundSW : chrome.runtime.sendMessage (bidirectional) - Options ↔ LocalStorage : Save/Load config - ContextMenu → BackgroundSW : Trigger action - ContentScript ↔ BackgroundSW : chrome.tabs.sendMessage (bidirectional) -

ContentScript → Scanner : Scan DOM - Scanner → Parser : Parse links - BackgroundSW → BobaAPIClient : HTTP requests - BobaAPIClient → Shared : API calls - ContentScript → SessionStorage : Store discovered - ContentScript → WebPages : Inject detection UI - BobaAPIClient → FastAPI : REST API :7187 - BobaAPIClient → GoService : REST API :7189 - FastAPI → qBitTorrent : Proxy operations - GoService → qBitTorrent : Direct API - qBitTorrent → qBitDB : Persist state - qBitTorrent → Trackers : Announce/Scrape - (drawio) WebPages → ContentScript : dashed (injection inbound)

3b) Module/file component diagram (#06) — code organization

8 packages, each with files + key functions (from .mmd): - **parser/**: magnetParser.js (parseMagnetURI(), extractInfoHash()), torrentParser.js (parseTorrentFile(), extractMetadata()), hashNormalizer.js (normalizeInfoHash(), validateHash()) - **scanner/**: domScanner.js (scanForLinks(), MutationObserver), linkExtractor.js (extractMagnetLinks(), extractTorrentLinks()), torrentBadge.js (injectBadge(), highlightTorrents()) - **api/**: BobaAPIClient.js (sendTorrent(), sendBatch(), healthCheck()), authManager.js (getAuthHeaders(), refreshToken()), retryHandler.js (exponentialBackoff(), circuitBreaker()) - **background/**: background.js (event listeners, message router), contextMenuHandler.js (createMenus(), handleClicks()), queueManager.js (enqueue(), processQueue(), retryFailed()), tabGroupProcessor.js (queryGroups(), processTabs()) - **content/**: content.js (initialize(), DOM scan trigger), contentUI.js (injectPanel(), showTorrentList()) - **popup/**: popup.js (renderTorrentList(), handleSend(), handleConfig()), popup.html, popup.css - **options/**: options.js (saveConfig(), loadConfig(), validateServer()), options.html - **shared/**: constants.js (MSG_TYPES, API_ENDPOINTS, DEFAULTS), utils.js (debounce(), throttle(), formatBytes()), storageAPI.js (getConfig(), setConfig(), getHistory())

Module dependencies (#06, both mmd+puml agree): -

scanner → parser (uses) - content → scanner (uses) - content → background (messaging, dashed) - popup → background (messaging, dashed) - background → api (uses) - background → shared (uses) - popup → shared (uses) - options → shared (uses) - api → shared (uses) - parser → shared (uses) - (mmd-only extra, likely typo) content → content (uses)

4) END-TO-END SEQUENCES

4a) Single Torrent Send (#04) — participants: User, Popup UI, Content Script (CS), Background SW (BG), BobaAPIClient (API), FastAPI :7187, Go Service :7189, qBitTorrent :8080

Detection phase: 1. User → CS: Browse torrent site 2. CS → CS: MutationObserver detects magnet: link in DOM 3. CS → CS: parser.normalizeURL() 4. CS → BG: chrome.runtime.sendMessage {type:"DISCOVERED_TORRENTS", torrents} 5. BG → Popup: chrome.runtime.sendMessage {type:"UPDATE_POPUP", torrents} 6. Popup → Popup: Render torrent list 7. Popup → User: Show discovered torrents

Send phase: 8. User → Popup: Click “Send” button 9. Popup → BG: chrome.runtime.sendMessage {type:"SEND_TORRENT", torrent} 10. BG → API: sendTorrent(torrent) 11. API → API: Build request with auth headers — **X-Api-Key + Basic Auth**

Two paths (mmd par / puml alt — both backends exercised): - *FastAPI path:* API → FastAPI POST /api/torrents/add (JSON body) → FastAPI validates torrent data → FastAPI → qBit POST /api/v2/torrents/add (form-data: urls, category) → qBit adds to BitTorrent engine → qBit → FastAPI 200 OK + {hash} → FastAPI → API 200 OK + {hash, status} - *Go Service path:* API → GoSvc POST /api/torrents/add (JSON body) → GoSvc validate + metadata lookup → GoSvc → qBit POST /api/v2/torrents/add (form-data) → qBit adds to engine → qBit → GoSvc 200 OK + {hash} → GoSvc → API 200 OK + {hash, status, metadata}

Completion: 12. API → BG: Promise resolved {success, hash, status} 13. BG → BG: Store in send_history; Update badge count 14. BG → Popup: chrome.runtime.sendMessage {type:"SEND_COMPLETE", result} 15. Popup → User: ✓ Notification “Torrent added” - Note (right of qBit): magnet links resolved via DHT for metadata.

4b) Tab Group Batch Send (#05) — participants: User, Context Menu, Background SW (BG), Tab Groups API (TGAPI = chrome.tabGroups), Tabs API (TabsAPI = chrome.tabs), Content Scripts (CS), BobaAPIClient (API), qBitTorrent :8080

1. User → Menu: Right-click on Tab Group
2. Menu → User: Show context menu “Send all torrents to qBit”
3. User → Menu: Select batch send option
4. Menu → BG: contextMenus.onClicked {menuItemId, parentMenuItemId}

5. BG → TGAPI: `chrome.tabGroups.query {title: groupName}`
6. TGAPI → BG: Return group objects[] with groupId
7. BG → TabsAPI: `chrome.tabs.query {groupId: gid}`
8. TabsAPI → BG: Return tabs[] with tabId, url

Loop — for each tab in group: 9. BG → CS:

`chrome.scripting.executeScript {target:{tabId}, files: ["content.js"]}` 10. CS → CS: Scan DOM for torrents 11. CS → CS: `parser.normalizeURLs()` 12. CS → BG: Return [{torrents:[...]}] (message RETURN_TORRENTS per #03)

Batch send: 13. BG → BG: Collect all torrents[] — **Flatten & deduplicate** 14. BG → API: `sendBatch(allTorrents)` 15. API → API: Build batch request with auth headers 16. API → qBit: `POST /api/v2/torrents/add (urls=magnet1\nmagnet2\n...)` — note: mmd #05 hits qBit directly; data-flow #03 routes through `POST /api/torrents/batch` on Boba Services first 17. qBit → qBit: Process each torrent in parallel 18. qBit → API: `200 OK + results[]` 19. API → BG: `{added: N, failed: [...]}` 20. BG → BG: Store results in `send_history` 21. BG → User: `chrome.notifications` “N torrents added successfully”

Tab-groups data-flow detail (#03): message types `DISCOVERED_TORRENTS / RETURN_TORRENTS`; batch endpoint `POST /api/torrents/batch` (JSON array of magnets) → proxied to qBit `/api/v2/torrents/add` (multiple urls) → result `200 OK + results[]` → notification “X torrents added”.

5) STATE MACHINES

5a) Offline-queue state machine (#09)

States: Idle, Pending, Sending, Queued, Retry, Completed, Failed (+ initial/final [*]).

Transitions (source → target : guard/trigger): - [*] → Idle : Extension starts - Idle → Pending : Torrent detected - Pending → Sending : Online detected - Pending → Queued : Offline detected - Pending → Retry : Send failed - Sending → Completed : 200 OK - Sending → Failed : 4xx/5xx error - Sending → Retry : Timeout - Queued → Pending : Back online - Queued → Failed : Max age exceeded - Retry → Sending : Retry attempt - Retry → Failed : Max retries exceeded - Retry → Queued : Still offline - Completed → [*] : Cleanup after TTL - Failed → Pending : User retries - Failed → [*] : Abandoned

State notes: - Pending: “Torrent detected in DOM / Awaiting send trigger” - Sending: “HTTP request in flight / BobaAPIClient active” - Queued: “Device offline / Stored for later retry” - Retry:

“Exponential backoff 1s → 2s → 4s → 8s” - Completed: “200 OK received / Added to send_history” - Failed: “Irrecoverable error / Logged for review”

5b) Extension lifecycle (#08) — flow (not a formal state machine, a staged lifecycle)

Stages (in order): Install (from store or .crx) → Grant (permissions) → Configure (server settings) → Discover (auto-discover server) → Browse (torrent sites) → Detect (torrent links) → Send (to qBitTorrent) → Monitor (downloads) → loops back to Browse.

Stage transitions w/ guards: - Install → Grant : `chrome.runtime.onInstalled` - Grant → Configure : Permissions granted (storage, scripting, tabs) - Configure → Discover : Save settings `chrome.storage.local` - Discover → Browse : Health check passed / server reachable - Browse → Detect : DOM contains torrent links on page - Detect → Send : Torrents discovered / shown in popup - Send → Monitor : Send successful / 200 OK received - Monitor → Browse : Continue browsing

Chrome events feeding lifecycle: - `chrome.runtime.onInstalled` → Install - `chrome.runtime.onStartup` → Browse (Restore config from storage) - `chrome.runtime.onUpdateAvailable` → Configure (Show changelog / migrate data)

Per-stage user actions: A1 Grant host/storage/scripting permissions (Install+Grant); A2 Enter server URL, API key, qBit URL (Configure); A3 Test connection / auto-detect Docker (Discover); A4 Navigate to torrent sites (Browse); A5 Review detected torrents in popup (Detect); A6 Click send / batch send / auto-send (Send); A7 Check progress in popup & qBit WebUI (Monitor).

6) SECURITY CONTROLS + TRUST BOUNDARIES (#10)

Trust boundaries (subgraphs): **Browser Environment, Network Layer, Boba Services, qBitTorrent.**

Browser Environment controls: - Extension Permissions (manifest.json): host, storage, scripting, tabs, contextMenus - *Content Security Policy (CSP)*: `script-src: 'self'` (no inline scripts); `connect-src: *` (API endpoints only); `object-src: 'none'` (no plugins) - *Credential Storage*: `chrome.storage.local` — **AES-256-GCM encrypted**; `chrome.storage.session` — non-

sensitive cache (plain) - *Crypto Operations*: Offscreen Document — Web Crypto API, SubtleCrypto.encrypt; PBKDF2 Key Derivation — user password → key

Network Layer controls: HTTPS Only (TLS 1.2+ required); Certificate Validation (no self-signed certs / cert pinning)

Boba Services controls: API Key Auth (X-API-Key header); Basic Auth (username:password); Rate Limiting (100 req/min per key)

qBitTorrent controls: WebUI Security (Cookie-based SID, CSRF protection); IP Filtering (allowlist mode)

Control flow / boundary crossings (#10): ExtPermissions → CSP → StorageSec → Crypto → (Encrypted payload) HTTPS → (Mutual TLS) BobaServices; HTTPS → (HTTPS + Cookie) qBitTorrent; BobaServices → Auth → RateLimit → (Proxied request) qBitTorrent.

Additional security notes from #01 legend: HTTPS for all external comms | credentials in chrome.storage.local | offscreen document for crypto | retry queue for offline | rate limiting | CSP headers | permission validation for all API calls.

7) BUILD PIPELINE STAGES (#11)

Linear stages: **Code** → **Lint** → **Test** → **Build** → **Package** → **Deployments**. - Code: Git repository, src/ directory - Lint: ESLint + Prettier, style checks - Test: Jest unit tests, coverage > 80% - Build: Webpack/Vite, bundle + minify - Package: Extension ZIP, manifest validation

Stage transitions (with gates): - Code → Lint : git push - Lint → Test : Pass | Lint → Code : Fail - Test → Coverage : Pass - Coverage → Build : ≥ 80% | Coverage → Code : < 80% - Build → ManifestValid : Bundle OK - ManifestValid → Package : Valid - Package → ChromeStore : Auto-deploy - Package → AMO : Auto-deploy - Package → Opera : Manual - Package → Edge : Auto-deploy - Package → Yandex : Manual

Deployment targets: Chrome Web Store (\$5 one-time fee, review 1-3 days); Firefox AMO (free, 1-7 days); Opera Addons (free, 2-5 days); Edge Addons (free, 1-5 days); Yandex Browser (free, 3-7 days).

CI/CD — GitHub Actions triggers: on: push / on: pull_request → Lint; on: release published → Build; on: schedule (weekly dependency check) → Test. > NOTE: contradicts Boba root constitution “NO CI/CD pipelines / no .github/workflows” — see open questions.

Quality Gates: Coverage $\geq 80\%$; 0 ESLint errors; Valid manifest.json; Package < 5MB.

8) COMPATIBILITY MATRIX (#12 — full, every cell)

Legend: ✓ Full (native) | △ Partial (polyfill/workaround) | ✗ No (not supported).

Feature	Chrome	Firefox	Opera	Edge	Yandex
Core Torrent Detection	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full
Tab Group Batch Send	✓ Full	△ Partial	✓ Full	✓ Full	✓ Full
Context Menu Actions	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full
Service Worker	✓ MV3	△ MV2	✓ MV3	✓ MV3	✓ MV3
Offscreen Document	✓ Full	✗ N/A	✓ Full	✓ Full	✓ Full
chrome.storage.session	✓ Full	✗ No	✓ Full	✓ Full	✓ Full
chrome.scripting API	✓ Full	△ Polyfill	✓ Full	✓ Full	✓ Full
chrome.notifications	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full
Auto-Update	✓ Store	✓ Store	✓ Store	✓ Store	△ Manual
Credential Encryption	✓ Full	△ Fallback	✓ Full	✓ Full	✓ Full
Background Message	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full
Badge Counter	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full
Popup UI	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full
Options Page	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full
Content Script Injection	✓ Full	✓ Full	✓ Full	✓ Full	✓ Full

Browser-specific notes: - **Firefox:** Tab Groups API not available (uses tab query fallback). Offscreen Document not supported (uses background-page crypto via Web Crypto API). storage.session replaced with localStorage fallback. MV2 uses background page instead of service worker. scripting API uses browser.tabs.executeScript via webextension-polyfill. All core features work with polyfills. - **Yandex:** Requires Opera extension package format. Auto-update must be checked manually via Yandex extension store. All Chromium APIs fully supported. - **Opera/Edge:** Fully Chromium compatible. Edge uses same package as Chrome. Opera requires manifest key in package. - **Chrome:** Reference implementation platform. All MV3 APIs used natively. No polyfills required.

9) OPEN QUESTIONS / INCONSISTENCIES

1. **CI/CD contradiction:** #11 build pipeline mandates GitHub Actions (on: push, on: release, on: schedule) and “Auto-deploy” to stores. Boba root CLAUDE.md/Constitution Hard-Stop #1 forbids ALL CI/CD pipelines and .github/workflows/*. The plan MUST reconcile — likely the extension deploy pipeline runs as a manual script (.ci.sh-style), not GitHub Actions.
2. **Single-send routing ambiguity:** #04 single-send has the API client call BOTH FastAPI (POST /api/torrents/add) AND Go Service (POST /api/torrents/add) — modeled as parallel/alt. Is this an either-or (one configured backend) or genuinely dual-write? #02 shows FastAPI getting POST /api/torrents/add and Go getting POST /api/torrents/batch — inconsistent endpoint→service mapping vs #04.
3. **Batch endpoint path inconsistency:** #03/#02 route batch through Boba POST /api/torrents/batch; #05 sequence has BG→API→qBit directly via POST /api/v2/torrents/add, skipping the Boba batch endpoint. Which is authoritative?
4. **Boba port reality vs diagrams:** diagrams use qBit WebUI port **8080**; the real Boba repo (per CLAUDE.md) uses qBittorrent on **7185** proxied via **7186**, merge service **7187**, boba-jackett **7189**. FastAPI=7187 and Go=7189 roughly align, but qBit 8080 does not match the repo’s 7185/7186. Plan must map BobaLink endpoints onto real Boba services.
5. **extension_config ↔ discovered_torrents relationship** is labeled “references config” but discovered_torrents has no config_id FK field — relationship is conceptual, not a declared FK. Confirm whether a FK column is needed.
6. **search_cache orphan:** no relationship; which service populates it (FastAPI search?) and where is it stored (chrome.storage? IndexedDB?) — storage backend for all 6 entities is unspecified (extension uses chrome.storage.local/session per #01; ER

implies relational/SQL types). Need a decision: chrome.storage vs IndexedDB vs server-side DB.

7. **Auth model:** #04 says X-API-Key + Basic Auth together; #10 lists API Key Auth and Basic Auth as separate controls. Confirm both are sent on every request vs per-service.
8. **source_tab_id, last_retry_at, last_health_check, magnet_uri/qbittorrent_hash/tags (send_history)** present in .mmd but dropped in .drawio — treat .mmd as authoritative (done above), but confirm the dropped fields are intended.
9. **content→content self-dependency (#06 mmd line 56)** is almost certainly a typo (should be content→shared or similar). Ignore unless intentional.
10. **MV2/MV3 dual-target:** matrix requires Firefox MV2 (background page) + Chromium MV3 (service worker). Build pipeline must produce two manifest variants + webextension-polyfill — not shown as a build stage in #11.